COMP 8005

Assignment 4

Report

Manraj Bhullar

A01018465

March 26, 2026

Overview

This report evaluates the scaling behavior of the distributed password cracking system by analyzing performance results from one to five workers. The goal of the experiments was to see if cracking time decreases as more workers are added and how this affects total runtime and speedup. Measurements were collected for each worker count and include compute time, end-to-end runtime, and latency measurements. The results are used to observe scaling behavior and evaluate how distributing work across multiple machines compares to the previous implementation, where performance eventually plateaued or even decreased once the single worker machine reached a thread count which was no longer efficient. After running experiments for up to three workers, a prediction is made for what will occur when the system is run with five workers. There is also a PCAP analysis at the end of the report.

Experiment Methodology

The controller and workers were run on separate hosts to mimic a distributed system and test worker scaling. Each worker was run using **3 threads** so that only the number of workers changed between tests. This thread count was chosen because it matched the lowest machine's available CPU cores minus one, ensuring consistent worker performance across all machines.

All experiments were performed using the **SHA512** hashing algorithm as the main test algorithm. The three-character password used for all trials was **"MI5"**. Only one password was chosen so the number of candidate attempts remains consistent across every test. The controller divided the search space into chunks and workers repeatedly requested new chunks until the password was found.

Each test was run with a chunk size of **20,000**. This value was chosen to balance work across all workers since the machines used were not identical in performance. A chunk size that is too large could cause a slower worker to delay completion, while a chunk size that is too small can increase overhead due to frequent chunk assignments and communication.

The test was ran five times for each worker count. The average of those five runs is used as the final reported result for that worker count. Multiple trials were used to reduce variation caused by scheduling differences.

Configuration:

- *Threads Per Worker:* 3
- *Hashing Algorithm:* SHA512
- *Test Password:* MI5
- *Chunk Size:* 20,000

Experiment Results

Check “data/experiments.xlsx” for the excel file.

Experiments

1 Worker (SHA512)

Run	Compute	Speed	Parse	Send	Return	E2E
1	58.85	4151.73	0.01	13.06	13.20	58.87
2	58.84	4152.34	0.01	11.95	14.80	58.86
3	58.86	4151.01	0.01	11.65	12.88	58.88
4	58.90	4147.13	0.01	11.83	14.04	58.93
5	58.79	4155.88	0.01	11.60	11.70	58.81
AVG	58.85	4151.62	0.01	12.02	13.32	58.87
MIN	58.79	4147.13	0.01	11.60	11.70	58.81
MAX	58.90	4155.88	0.01	13.06	14.80	58.93
SD	0.04	3.13	0.00	0.60	1.18	0.04

2 Workers (SHA512)

Run	Compute	Speed	Parse	Send	Return	E2E
1	39.70	6075.87	0.01	9.69	14.60	39.72
2	39.69	6084.10	0.01	11.10	11.10	39.71
3	40.17	6053.05	0.01	9.76	12.03	40.20
4	40.26	6047.60	0.01	8.81	10.85	40.28
5	39.71	6082.27	0.01	10.67	12.28	39.74
AVG	39.90	6068.58	0.01	10.01	12.17	39.93
MIN	39.69	6047.60	0.01	8.81	10.85	39.71
MAX	40.26	6084.10	0.01	11.10	14.60	40.28
SD	0.29	17.05	0.00	0.90	1.49	0.28

3 Workers (SHA512)

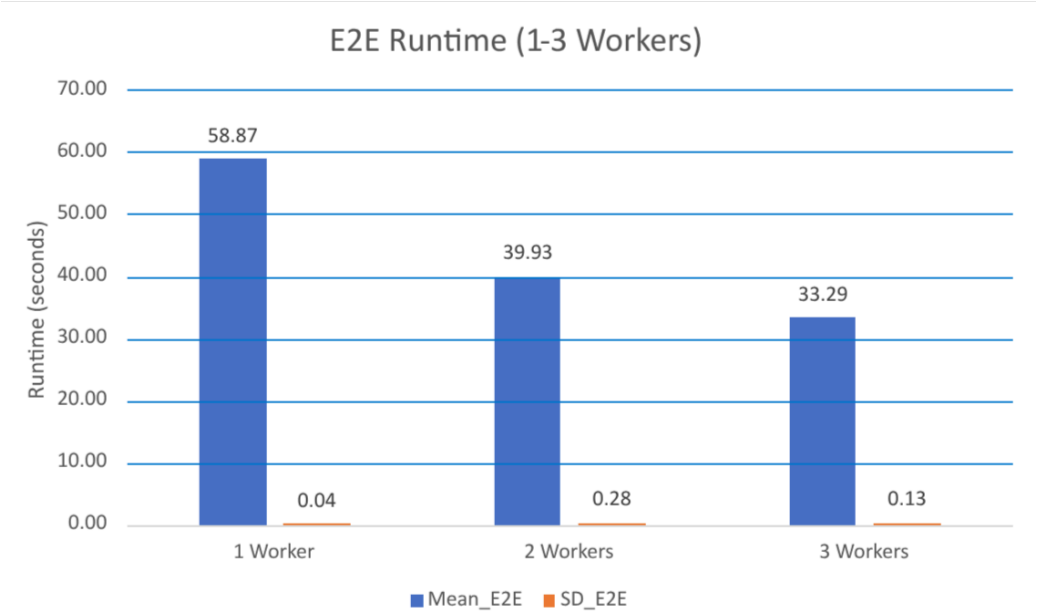
Run	Compute	Speed	Parse	Send	Return	E2E
1	33.36	7454.63	0.01	8.49	10.80	33.38
2	33.14	7456.94	0.01	8.07	14.66	33.16
3	33.23	7467.75	0.01	9.52	13.49	33.25
4	33.16	7492.49	0.01	8.11	10.56	33.18
5	33.45	7450.79	0.01	10.19	14.78	33.47
AVG	33.27	7464.52	0.01	8.88	12.86	33.29

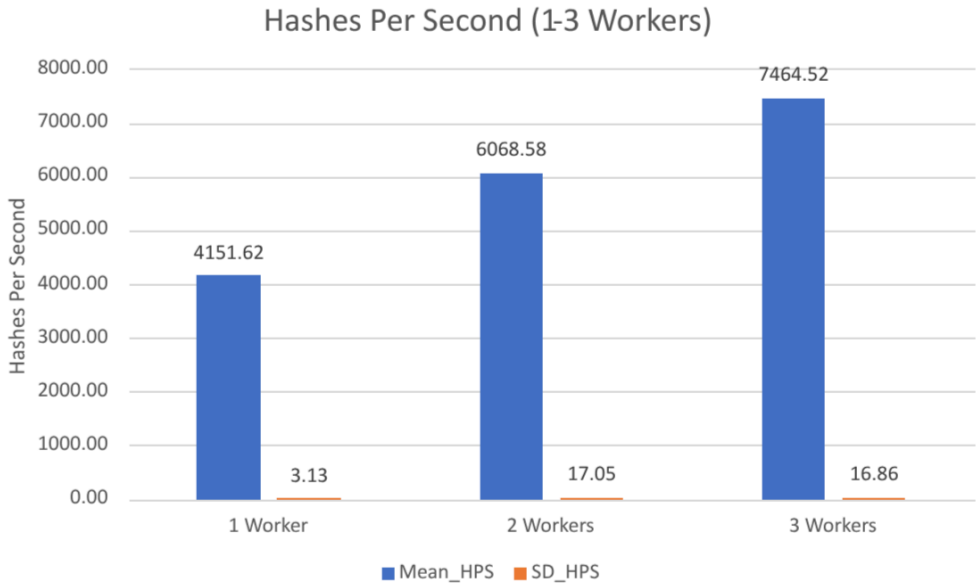
MIN	33.14	7450.79	0.01	8.07	10.56	33.16
MAX	33.45	7492.49	0.01	10.19	14.78	33.47
SD	0.13	16.86	0.00	0.94	2.05	0.13

5 Workers (SHA512)

Run	**Compute**	**Speed**	**Parse**	**Send**	**Return**	**E2E**
1	23.61	9805.50	0.01	7.90	12.89	23.63
2	23.15	9837.72	0.01	11.03	10.99	23.17
3	23.16	9798.11	0.01	11.80	56.17	23.23
4	23.31	9746.36	0.01	9.24	10.10	23.33
5	23.65	9747.77	0.03	10.54	11.45	23.67
AVG	23.38	9787.09	0.01	10.10	20.32	23.41
MIN	23.15	9746.36	0.01	7.90	10.10	23.17
MAX	23.65	9837.72	0.03	11.80	56.17	23.67
SD	0.24	39.46	0.01	1.54	20.07	0.23
Unit	*seconds*	*hashes/sec*	*milliseconds*	*milliseconds*	*milliseconds*	*seconds*

Observation After 3 Workers





- Performance increases as workers are added which confirms distributed scaling improves cracking rate.
- The improvement is not perfectly linear and depends on the capability of the worker added.
- The first machine is the fastest (~4350 HPS), while the other two are slower (~1975 HPS and ~1375 HPS), which explains the uneven gains.
- Adding a similar speed machine as the first worker would give even larger gains.
- Slower workers still contribute but increase throughput less than faster machines.
- This is much faster than the last implementation using a single multi-threaded worker where a plateau was hit after a certain thread count.

Prediction for 5 Workers

We can use simple throughput projection to predict performance at 5 workers. From 1 to 3 workers, throughput increased but not perfectly linear. The jump from 1 to 2 workers was larger than from 2 to 3 workers, which reflects the different speeds of the machines used.

1 Worker: 4151.62 hashes/sec
3 Workers: 7464.52 hashes/sec

Speedup @ 3 Workers = 7464.52 / 4151.62 ≈ 1.8x

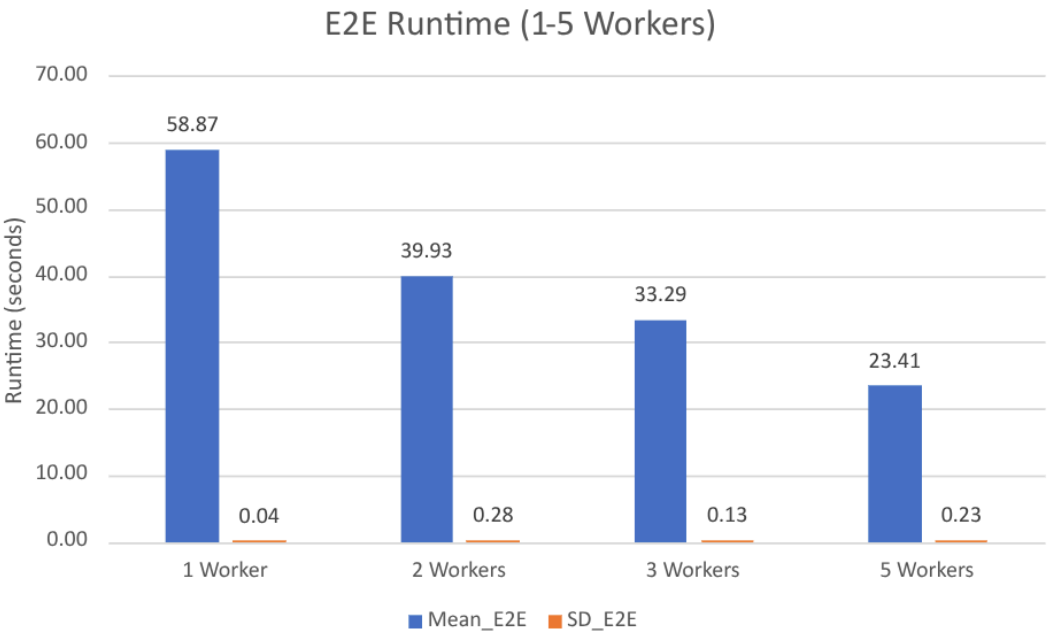
Average gain per added worker (from 1 to 3 workers):

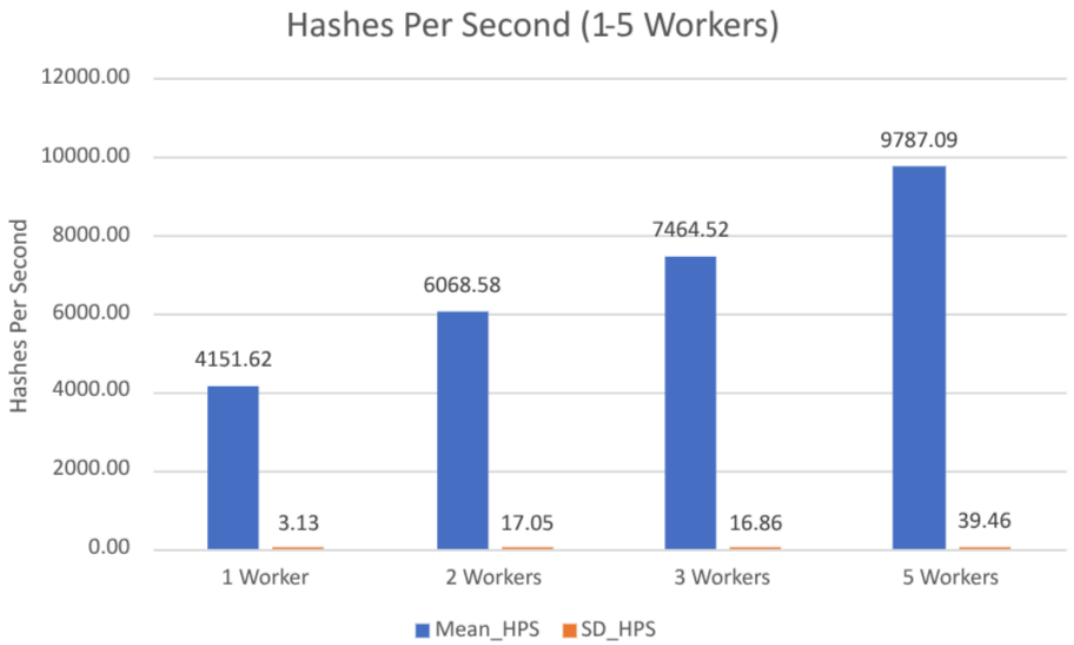
- Increase = 7464.52 − 4151.62 = 3312.90 HPS
- Workers added = 2
- Gain per worker ≈ 3312.90 / 2 = 1656.45 HPS

Assuming the next two workers are similar in speed to Worker 2 and Worker 3, the two new workers together would add about 3300 HPS.

Predicted Performance @ 5 Workers ≈ 7464.52 + 3300 = 10,764.52 hashes/sec

Observation After 5 Workers

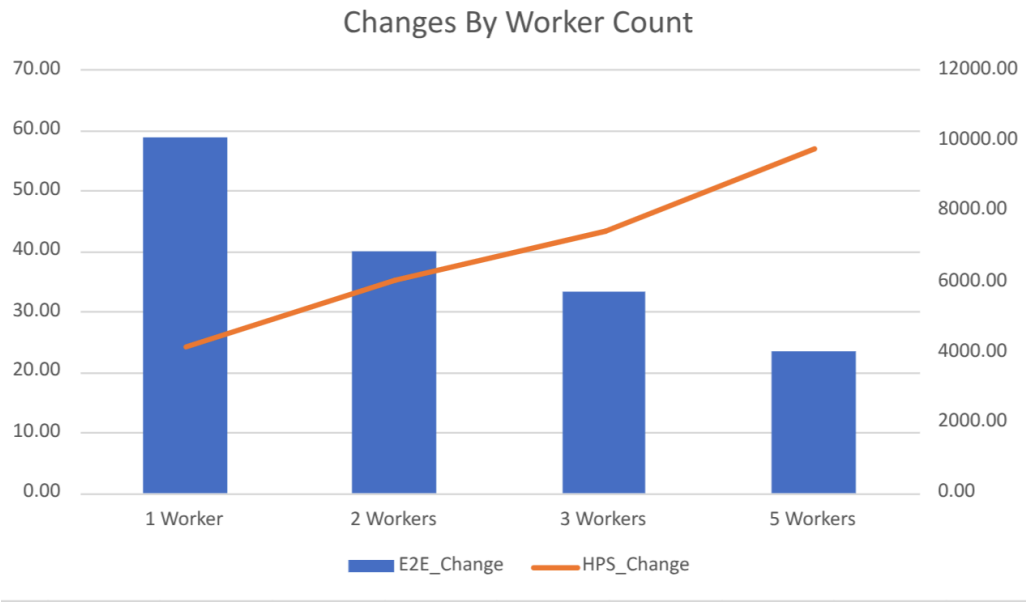




- Predicted throughput at 5 workers was about 10.8k HPS based on scaling from 1 to 3 workers.
- Actual throughput reached 9787 HPS, about 1000 HPS lower than predicted.
- The prediction assumed each new worker would add about 1656 HPS, but the actual added workers averaged only about (1900 + 850) / 2 ≈ 1375 HPS each.
- Because the new workers were slower on average than the assumed level for prediction, the total increase was smaller than expected.
- The prediction overestimated performance but still correctly showed throughput would still continue increasing even if machines vary in speed.

Individual Worker Speeds

Worker	HPS	Threads
Worker 1	4350	3
Worker 2	1975	3
Worker 3	1375	3
Worker 4	1900	3
Worker 5	850	3



- Each increase in total throughput aligns with the speed of the machines that were added.
- The jump to two workers is large because Worker 2 performs around 1975 HPS.
- The next increase is smaller since Worker 3 runs at about 1375 HPS.
- The final jump from three to five workers reflects the combined contribution of Worker 4 (~1900 HPS) and Worker 5 (~850 HPS).
- Overall performance changes depend on the hashing capability of the added machines rather than just the number of workers.

Final Thoughts:

These results make sense because the total performance followed how fast each machine was and not just how many workers were running. The uneven jumps happened because the machines had different speeds. This is likely why datacenters use many similar machines since scaling becomes easier to predict. Different machines can still work well as shown in this experiment, but it would help to know how fast each machine is beforehand. An example of this would be by having a worker send a small benchmark of its performance during registration so the work assigned to it can be adjusted individually by the controller instead of using the same chunk size for all. The controller could also use the heartbeat statistics to adjust chunk sizes over time. This would help prevent machines from being

underloaded or overloaded. I ran these tests using my own machines but i suspect that running them in the BCIT lab where the machines are identical would produce more linear gains.

PCAP Analysis

There are packet captures in the *'pcap'* folder for a single worker and multi-worker experiment. The screenshot below is from a run with two workers.

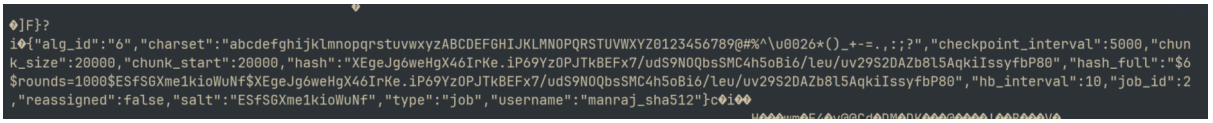
PCAP File: *pcap/two_workers.pcap*

```
…/src/controller main ● ? ❱ tcpdump -r two_workers.pcap -nn
reading from file two_workers.pcap, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
Warning: interface names might be incorrect
03:16:03.355929 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [S], seq 2148430761, win 64240, options [mss 1460,sackOK,TS val 1847
889868 ecr 0,nop,wscale 10], length 0
03:16:03.355970 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [S.], seq 80286900, ack 2148430762, win 65160, options [mss 1460,sac
kOK,TS val 3000764258 ecr 1847889868,nop,wscale 10], length 0
03:16:03.359796 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [P.], seq 1:5, ack 1, win 63, options [nop,nop,TS val 1847889872 ecr
 3000764258], length 4
03:16:03.359797 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [.], ack 1, win 63, options [nop,nop,TS val 1847889872 ecr 300076425
8], length 0
03:16:03.359797 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [P.], seq 5:24, ack 1, win 63, options [nop,nop,TS val 1847889872 ec
r 3000764258], length 19
03:16:03.359896 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [.], ack 5, win 64, options [nop,nop,TS val 3000764262 ecr 184788987
2], length 0
03:16:03.359927 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [.], ack 5, win 64, options [nop,nop,TS val 3000764262 ecr 184788987
2], length 0
03:16:03.359934 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [.], ack 24, win 64, options [nop,nop,TS val 3000764262 ecr 18478898
72], length 0
03:16:03.360395 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [P.], seq 1:5, ack 24, win 64, options [nop,nop,TS val 3000764262 ec
r 1847889872], length 4
03:16:03.360427 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [P.], seq 5:87, ack 24, win 64, options [nop,nop,TS val 3000764262 e
cr 1847889872], length 82
03:16:03.362758 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [.], ack 5, win 63, options [nop,nop,TS val 1847889876 ecr 300076426
2], length 0
03:16:03.362758 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [.], ack 87, win 63, options [nop,nop,TS val 1847889876 ecr 30007642
62], length 0
03:16:03.362759 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [P.], seq 24:28, ack 87, win 63, options [nop,nop,TS val 1847889876
ecr 3000764262], length 4
03:16:03.362759 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [P.], seq 28:66, ack 87, win 63, options [nop,nop,TS val 1847889876
ecr 3000764262], length 38
03:16:03.362827 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [.], ack 66, win 64, options [nop,nop,TS val 3000764265 ecr 18478898
76], length 0
03:16:03.362939 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [P.], seq 87:91, ack 66, win 64, options [nop,nop,TS val 3000764265
ecr 1847889876], length 4
03:16:03.362948 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [P.], seq 91:606, ack 66, win 64, options [nop,nop,TS val 3000764265
 ecr 1847889876], length 515
03:16:03.367971 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [.], ack 606, win 63, options [nop,nop,TS val 1847889881 ecr 3000764
265], length 0
03:16:03.367971 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [P.], seq 66:70, ack 606, win 63, options [nop,nop,TS val 1847889881
 ecr 3000764265], length 4
03:16:03.367971 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [P.], seq 70:99, ack 606, win 63, options [nop,nop,TS val 1847889881
 ecr 3000764265], length 29
03:16:03.368004 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [.], ack 99, win 64, options [nop,nop,TS val 3000764270 ecr 18478898
81], length 0
03:16:03.773627 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [P.], seq 99:103, ack 606, win 63, options [nop,nop,TS val 184789028
5 ecr 3000764270], length 4
03:16:03.773627 wlan0 In  IP 192.168.68.51.47340 > 192.168.68.75.8000: Flags [P.], seq 103:151, ack 606, win 63, options [nop,nop,TS val 18478902
85 ecr 3000764270], length 48
03:16:03.773670 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.51.47340: Flags [.], ack 151, win 64, options [nop,nop,TS val 3000764676 ecr 1847890
285], length 0
03:16:03.782177 wlan0 In  IP 192.168.68.77.48636 > 192.168.68.75.8000: Flags [S], seq 3875177434, win 64240, options [mss 1460,sackOK,TS val 1057
647068 ecr 0,nop,wscale 7], length 0
03:16:03.782211 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.77.48636: Flags [S.], seq 566153952, ack 3875177435, win 65160, options [mss 1460,sa
ckOK,TS val 3848095345 ecr 1057647068,nop,wscale 10], length 0
03:16:03.788815 wlan0 In  IP 192.168.68.77.48636 > 192.168.68.75.8000: Flags [P.], seq 1:5, ack 1, win 502, options [nop,nop,TS val 1057647075 ec
r 3848095345], length 4
03:16:03.788816 wlan0 In  IP 192.168.68.77.48636 > 192.168.68.75.8000: Flags [.], ack 1, win 502, options [nop,nop,TS val 1057647075 ecr 38480953
45], length 0
03:16:03.788816 wlan0 In  IP 192.168.68.77.48636 > 192.168.68.75.8000: Flags [P.], seq 5:24, ack 1, win 502, options [nop,nop,TS val 1057647075 e
cr 3848095345], length 19
03:16:03.788870 wlan0 Out IP 192.168.68.75.8000 > 192.168.68.77.48636: Flags [.], ack 5, win 64, options [nop,nop,TS val 3848095352 ecr 105764707
5], length 0
```

- This packet capture confirms the controller at 192.168.68.75 running on port 8000 is coordinating the cracking process correctly.
- The worker host machines are from 192.168.68.51 and 192.168.68.77
- This capture was ran using two workers and a checkpoint interval of 5,000 and chunk size of 20,000.

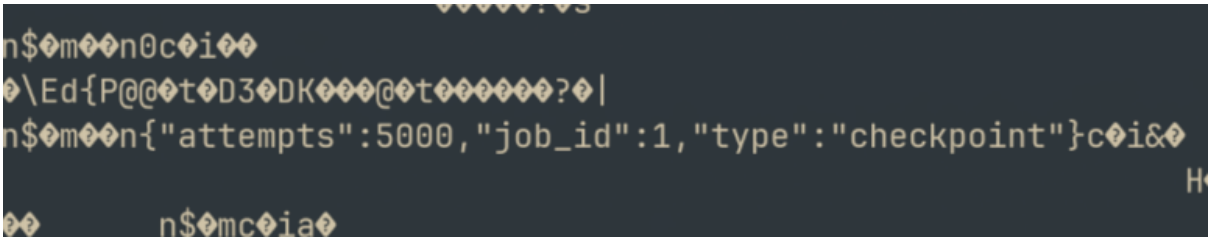
- In addition to the regular communication, the controller now receives all checkpoint updates from the two workers. You can see all outgoing checkpoints from the workers to the controller within the capture.

Checkpoint Interval Gets Sent with Job Data to Worker



- This is raw data of the pcap printed with *cat*

Checkpoint Sent to Controller



- This is raw data of the pcap printed with *cat*